Experiment No. 1 — Data Wrangling I

Step 1 — Import All Required Python Libraries

In [5]:
```
# Core libraries
import os
import numpy as np
import pandas as pd

# Visualization libraries
import matplotlib.pyplot as plt
import seaborn as sns

# Machine Learning / Preprocessing
from sklearn import preprocessing
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, MinMaxScaler

# Display settings
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 200)
```

Step 2 — Dataset Description & Source

Dataset: Iris Flower Dataset
Source: Kaggle — https://www.kaggle.com/datasets/uciml/iris
Original Source: UCI Machine Learning Repository

Description:
The Iris dataset was introduced by British statistician R.A. Fisher in his 1936 paper "*The Use of Multiple Measurements in Taxonomic Problems*". It contains measurements of 150 iris flowers from 3 different species:

- *Iris-setosa*
- *Iris-versicolor*
- *Iris-virginica*

Each species has **50 samples**. The dataset has **6 columns**:

Column	Type	Description
Id	Integer	Unique identifier
SepalLengthCm	Float	Length of sepal in cm
SepalWidthCm	Float	Width of sepal in cm
PetalLengthCm	Float	Length of petal in cm

Column	Type	Description
PetalWidthCm	Float	Width of petal in cm
Species	String (Categorical)	Iris species name

Step 3 — Load the Dataset into a Pandas DataFrame

> 📥 **Download the dataset:**
> Go to: https://www.kaggle.com/datasets/uciml/iris
> Download `Iris.csv` and place it in the **same folder as this notebook**.
> The file should be named `Iris.csv`.

In [6]:
```
# Load the dataset
# Make sure Iris.csv is in the same directory as this notebook
df = pd.read_csv('Iris.csv')

print("Dataset loaded successfully!")
print(f"   Shape: {df.shape[0]} rows × {df.shape[1]} columns\n")

# Display first 5 rows
print(" First 5 rows (df.head())")
df.head()
```

```
Dataset loaded successfully!
   Shape: 150 rows × 6 columns

 First 5 rows (df.head())
```

Out[6]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

In [7]:
```
# Display last 5 rows
print("--- Last 5 rows (df.tail()) ---")
df.tail()
```

```
--- Last 5 rows (df.tail()) ---
```

Out[7]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
145	146	6.7	3.0	5.2	2.3	Iris-virginica
146	147	6.3	2.5	5.0	1.9	Iris-virginica
147	148	6.5	3.0	5.2	2.0	Iris-virginica
148	149	6.2	3.4	5.4	2.3	Iris-virginica
149	150	5.9	3.0	5.1	1.8	Iris-virginica

Step 4 — Data Preprocessing

4a. DataFrame Basic Info

In [8]:

```
# Index of the dataframe
print("Index:", df.index)

# Column labels
print("\nColumns:", df.columns.tolist())

# Shape (rows, columns)
print("\nShape:", df.shape)

# Column names as array
print("\nColumn values array:", df.columns.values)
```

```
Index: RangeIndex(start=0, stop=150, step=1)

Columns: ['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm',
'Species']

Shape: (150, 6)

Column values array: ['Id' 'SepalLengthCm' 'SepalWidthCm' 'PetalLengthCm' 'PetalW
idthCm'
 'Species']
```

In [9]:

```
# Data types of each column
print("--- Data Types (df.dtypes) ---")
print(df.dtypes)
```

```
--- Data Types (df.dtypes) ---
Id                 int64
SepalLengthCm    float64
SepalWidthCm     float64
PetalLengthCm    float64
PetalWidthCm     float64
Species           object
dtype: object
```

4b. Initial Statistics using describe()

In [10]:
```
# Statistical summary — numeric columns
print("--- Statistical Summary (numeric) ---")
df.describe()
```

```
--- Statistical Summary (numeric) ---
```

Out[10]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

In [11]:
```
# Statistical summary — all columns (including categorical)
print("--- Statistical Summary (all columns) ---")
df.describe(include='all')
```

```
--- Statistical Summary (all columns) ---
```

Out[11]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Sp
count	150.000000	150.000000	150.000000	150.000000	150.000000	
unique	NaN	NaN	NaN	NaN	NaN	
top	NaN	NaN	NaN	NaN	NaN	se
freq	NaN	NaN	NaN	NaN	NaN	
mean	75.500000	5.843333	3.054000	3.758667	1.198667	
std	43.445368	0.828066	0.433594	1.764420	0.763161	
min	1.000000	4.300000	2.000000	1.000000	0.100000	
25%	38.250000	5.100000	2.800000	1.600000	0.300000	
50%	75.500000	5.800000	3.000000	4.350000	1.300000	
75%	112.750000	6.400000	3.300000	5.100000	1.800000	
max	150.000000	7.900000	4.400000	6.900000	2.500000	

4c. Checking for Missing Values

In [12]:
```
# Check missing values in the entire dataframe (True = missing)
print("--- Missing Values in DataFrame (df.isnull()) ---")
df.isnull().head(10)
```

```
--- Missing Values in DataFrame (df.isnull()) ---
```

Out[12]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	False	False	False	False
3	False	False	False	False	False	False
4	False	False	False	False	False	False
5	False	False	False	False	False	False
6	False	False	False	False	False	False
7	False	False	False	False	False	False
8	False	False	False	False	False	False
9	False	False	False	False	False	False

In [13]:
```
# Is there any missing value per column?
print("--- Any missing values per column? (df.isnull().any()) ---")
print(df.isnull().any())
```

```
--- Any missing values per column? (df.isnull().any()) ---
Id               False
SepalLengthCm    False
SepalWidthCm     False
PetalLengthCm    False
PetalWidthCm     False
Species          False
dtype: bool
```

In [14]:
```
# Count of missing values per column
print("--- Count of missing values per column (df.isnull().sum()) ---")
print(df.isnull().sum())
print(f"\nTotal missing values in dataset: {df.isnull().sum().sum()}")
```

```
--- Count of missing values per column (df.isnull().sum()) ---
Id               0
SepalLengthCm    0
SepalWidthCm     0
PetalLengthCm    0
PetalWidthCm     0
Species          0
dtype: int64

Total missing values in dataset: 0
```

In [15]:
```
# Count missing values for a specific column
print("Missing values in 'SepalLengthCm':", df['SepalLengthCm'].isnull().sum())
print("Missing values in 'Species'      :", df['Species'].isnull().sum())
```

```
Missing values in 'SepalLengthCm': 0
Missing values in 'Species'      : 0
```

4d. DataFrame Slicing & Selection Operations

In [16]:
```
# Read a specific column
print("--- SepalLengthCm column ---")
print(df['SepalLengthCm'].head(6))
```

```
--- SepalLengthCm column ---
0    5.1
1    4.9
2    4.7
3    4.6
4    5.0
5    5.4
Name: SepalLengthCm, dtype: float64
```

In [17]:
```
# Sort by index labels (columns in descending order)
print("--- Sort columns in descending order ---")
df.sort_index(axis=1, ascending=False).head()
```

```
--- Sort columns in descending order ---
```

Out[17]:

	Species	SepalWidthCm	SepalLengthCm	PetalWidthCm	PetalLengthCm	Id
0	Iris-setosa	3.5	5.1	0.2	1.4	1
1	Iris-setosa	3.0	4.9	0.2	1.4	2
2	Iris-setosa	3.2	4.7	0.2	1.3	3
3	Iris-setosa	3.1	4.6	0.2	1.5	4
4	Iris-setosa	3.6	5.0	0.2	1.4	5

In [18]:
```
# Sort rows by column values
print("--- Sort by SepalLengthCm ---")
df.sort_values(by='SepalLengthCm').head()
```

```
--- Sort by SepalLengthCm ---
```

Out[18]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
13	14	4.3	3.0	1.1	0.1	Iris-setosa
8	9	4.4	2.9	1.4	0.2	Iris-setosa
42	43	4.4	3.2	1.3	0.2	Iris-setosa
38	39	4.4	3.0	1.3	0.2	Iris-setosa
41	42	4.5	2.3	1.3	0.3	Iris-setosa

In [19]:
```
# Integer location based selection — row at index 5
print("--- Row at position 5 (df.iloc[5]) ---")
print(df.iloc[5])
```

```
--- Row at position 5 (df.iloc[5]) ---
Id                          6
SepalLengthCm             5.4
SepalWidthCm              3.9
PetalLengthCm             1.7
PetalWidthCm              0.4
Species           Iris-setosa
Name: 5, dtype: object
```

In [20]:
```
# Slice first 3 rows
print("--- First 3 rows (df[0:3]) ---")
df[0:3]
```

```
--- First 3 rows (df[0:3]) ---
```

Out[20]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa

In [21]:
```
# Select specific columns by label
print("--- Select SepalLengthCm & SepalWidthCm ---")
df.loc[:, ['SepalLengthCm', 'SepalWidthCm']].head()
```

```
--- Select SepalLengthCm & SepalWidthCm ---
```

Out[21]:

	SepalLengthCm	SepalWidthCm
0	5.1	3.5
1	4.9	3.0
2	4.7	3.2
3	4.6	3.1
4	5.0	3.6

In [22]:
```
# Subset: first 4 rows
print("--- First 4 rows (df.iloc[:4, :]) ---")
df.iloc[:4, :]
```

```
--- First 4 rows (df.iloc[:4, :]) ---
```

Out[22]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa

In [23]:
```
# Subset: first 3 columns
print("--- First 3 columns (df.iloc[:, :3]) ---")
df.iloc[:, :3].head()
```

```
--- First 3 columns (df.iloc[:, :3]) ---
```

Out[23]:

	Id	SepalLengthCm	SepalWidthCm
0	1	5.1	3.5
1	2	4.9	3.0
2	3	4.7	3.2
3	4	4.6	3.1
4	5	5.0	3.6

In [24]:
```
# Subset: first 4 rows and first 4 columns
print("--- First 4 rows × 4 columns (df.iloc[:4, :4]) ---")
df.iloc[:4, :4]
```

```
--- First 4 rows × 4 columns (df.iloc[:4, :4]) ---
```

Out[24]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm
0	1	5.1	3.5	1.4
1	2	4.9	3.0	1.4
2	3	4.7	3.2	1.3
3	4	4.6	3.1	1.5

Step 5 — Data Formatting & Data Normalization

5a. Check & Fix Data Types

In [25]:
```
# Summary of variable types
print("--- Variable Types Summary ---")
for col in df.columns:
    print(f"  {col:<20} → dtype: {df[col].dtype}")
```

```
--- Variable Types Summary ---
  Id                   → dtype: int64
  SepalLengthCm        → dtype: float64
  SepalWidthCm         → dtype: float64
  PetalLengthCm        → dtype: float64
  PetalWidthCm         → dtype: float64
  Species              → dtype: object
```

In [26]:
```
# Separate numeric and categorical columns
numeric_cols     = df.select_dtypes(include=[np.number]).columns.tolist()
categorical_cols = df.select_dtypes(include=['object']).columns.tolist()

print("Numeric columns    :", numeric_cols)
print("Categorical columns:", categorical_cols)
```

```
Numeric columns    : ['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'Pe
talWidthCm']
Categorical columns: ['Species']
```

5b. Min-Max Normalization (Data Normalization)

In [27]:
```
# Min-Max Normalization on SepalLengthCm column
x = df[['SepalLengthCm']].values.astype(float)

min_max_scaler = MinMaxScaler()
x_scaled       = min_max_scaler.fit_transform(x)
df_normalized  = pd.DataFrame(x_scaled, columns=['SepalLengthCm_normalized'])

print("--- Normalized SepalLengthCm (Min-Max Scaling) ---")
df_normalized.head(10)
```

```
--- Normalized SepalLengthCm (Min-Max Scaling) ---
```

Out[27]:

	SepalLengthCm_normalized
0	0.222222
1	0.166667
2	0.111111
3	0.083333
4	0.194444
5	0.305556
6	0.083333
7	0.194444
8	0.027778
9	0.166667

In [28]:
```
# Normalize ALL numeric columns (excluding Id)
feature_cols   = ['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthC
scaler         = MinMaxScaler()
df_all_norm    = df.copy()
df_all_norm[feature_cols] = scaler.fit_transform(df[feature_cols])

print("--- All Numeric Features Normalized ---")
df_all_norm[feature_cols].head(10)
```

```
--- All Numeric Features Normalized ---
```

Out[28]:

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
0	0.222222	0.625000	0.067797	0.041667
1	0.166667	0.416667	0.067797	0.041667
2	0.111111	0.500000	0.050847	0.041667
3	0.083333	0.458333	0.084746	0.041667
4	0.194444	0.666667	0.067797	0.041667
5	0.305556	0.791667	0.118644	0.125000
6	0.083333	0.583333	0.067797	0.083333
7	0.194444	0.583333	0.084746	0.041667
8	0.027778	0.375000	0.067797	0.041667
9	0.166667	0.458333	0.084746	0.000000

Step 6 — Turning Categorical Variables into Quantitative Variables

6a. Label Encoding

In [29]:

```
# Unique species before encoding
print("Unique values before encoding:", df['Species'].unique())

# Apply Label Encoding
df_label = df.copy()
label_encoder = LabelEncoder()
df_label['Species'] = label_encoder.fit_transform(df_label['Species'])

# Unique species after encoding
print("Unique values after encoding :", df_label['Species'].unique())
print("\nEncoding Map:")
for i, cls in enumerate(label_encoder.classes_):
    print(f"  {cls} → {i}")

print("\n--- Label Encoded DataFrame (first 10 rows) ---")
df_label.head(10)
```

```
Unique values before encoding: ['Iris-setosa' 'Iris-versicolor' 'Iris-virginica']
Unique values after encoding : [0 1 2]

Encoding Map:
  Iris-setosa → 0
  Iris-versicolor → 1
  Iris-virginica → 2

--- Label Encoded DataFrame (first 10 rows) ---
```

Out[29]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	0
1	2	4.9	3.0	1.4	0.2	0
2	3	4.7	3.2	1.3	0.2	0
3	4	4.6	3.1	1.5	0.2	0
4	5	5.0	3.6	1.4	0.2	0
5	6	5.4	3.9	1.7	0.4	0
6	7	4.6	3.4	1.4	0.3	0
7	8	5.0	3.4	1.5	0.2	0
8	9	4.4	2.9	1.4	0.2	0
9	10	4.9	3.1	1.5	0.1	0

6b. One-Hot Encoding (using sklearn)

In [30]:

```
# Apply One-Hot Encoding
df_ohe = df.copy()

# First apply label encoding to Species
df_ohe['Species'] = label_encoder.fit_transform(df_ohe['Species'])

# Remove Species column for features
features_df = df_ohe.drop(columns=['Species'])

# One-hot encode the Species column
enc       = OneHotEncoder(sparse_output=False)
enc_array = enc.fit_transform(df_ohe[['Species']])
enc_df    = pd.DataFrame(enc_array)

# Join encoded values back to features
df_encode = features_df.join(enc_df)

# Rename the new columns
df_encode.rename(columns={0.0: 'Iris-Setosa', 1.0: 'Iris-Versicolor', 2.0: 'Iris

print("--- One-Hot Encoded DataFrame ---")
df_encode.head()
```

```
--- One-Hot Encoded DataFrame ---
```

Out[30]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Iris-Setosa	Iris Versicolc
0	1	5.1	3.5	1.4	0.2	1.0	0.
1	2	4.9	3.0	1.4	0.2	1.0	0.
2	3	4.7	3.2	1.3	0.2	1.0	0.
3	4	4.6	3.1	1.5	0.2	1.0	0.
4	5	5.0	3.6	1.4	0.2	1.0	0.

6c. Dummy Variable Encoding (using pandas get_dummies)

In [31]:

```
# Dummy encoding with pandas get_dummies (k-1 variables, drop_first=True)
df_dummy = df.copy()
df_dummy = pd.get_dummies(df_dummy, columns=['Species'], drop_first=True)

print("--- Dummy Encoded DataFrame (k-1 dummy variables) ---")
df_dummy.head()
```

```
--- Dummy Encoded DataFrame (k-1 dummy variables) ---
```

Out[31]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species_Iris-versicolor	Spe
0	1	5.1	3.5	1.4	0.2	False	
1	2	4.9	3.0	1.4	0.2	False	
2	3	4.7	3.2	1.3	0.2	False	
3	4	4.6	3.1	1.5	0.2	False	
4	5	5.0	3.6	1.4	0.2	False	

In [32]:

```
# Full one-hot encoding using get_dummies (k dummy variables)
df_dummies_full = pd.get_dummies(df, columns=['Species'], drop_first=False)

print("--- Full One-Hot Encoding via get_dummies ---")
df_dummies_full.head()
```

```
--- Full One-Hot Encoding via get_dummies ---
```

Out[32]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species_Iris-setosa	Spe
0	1	5.1	3.5	1.4	0.2	True	
1	2	4.9	3.0	1.4	0.2	True	
2	3	4.7	3.2	1.3	0.2	True	
3	4	4.6	3.1	1.5	0.2	True	
4	5	5.0	3.6	1.4	0.2	True	